LAPORAN PRAKTIKUM

TEKNOLOGI IoT II

Modul 6

DISUSUN OLEH:

Nama : Salwa Fazirah
NPM : 25782054
Kelas :3B

Dosen Pengampu:

Dr. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027

1. Tujuan Praktikum

- Memahami konsep dasar dan arsitektur komunikasi *Publish-Subscribe* serta peran MQTT *Broker* dalam ekosistem IoT industri.
- Mampu mengonfigurasi pustaka `PubSubClient` dan `ArduinoJson` pada Arduino IDE untuk mikrokontroler NodeMCU ESP8266.
- Mampu menyusun (*serialize*) data pembacaan sensor suhu dan kelembapan (DHT11/DHT22) ke dalam format JSON dan mengirimkannya secara periodik tanpa memblokir sistem (*non-blocking timer*).
- Mampu menguraikan (*deserialize*) paket JSON melalui fungsi *callback* MQTT untuk mengendalikan aktuator LED secara asinkron.
- Mampu menerapkan fitur keandalan sistem industri seperti pemulihan koneksi otomatis (*auto-reconnect*) dan pesan status darurat *Last Will and Testament* (LWT).
- Mampu menguji dan menganalisis arus data dua arah menggunakan peranti lunak diagnostik industri seperti *MQTT Explorer*.

2. Dasar Teori

Komunikasi IoT skala besar (*Large-Scale IoT*) memerlukan protokol yang efisien dalam penggunaan *bandwidth* dan daya. Berbeda dengan HTTP yang berbasis *Request-Response*, **MQTT (Message Queuing Telemetry Transport)** menggunakan arsitektur **Publish-Subscribe** yang terpusat melalui *MQTT Broker*.

Karakteristik utama MQTT meliputi:

1. **Pemisahan (*Decoupling*):** Pemisahan ruang (*spatial*), waktu (*temporal*), dan sinkronisasi (*synchronization*) antara pengirim dan penerima.
2. **Hierarki Topik (*Topic Hierarchy*):** Pengalamatan pesan menggunakan string berjenjang (misalnya `kampus/labiot/nim/telemetri`) dengan dukungan *wildcard* (`+` dan `#`).
3. **QoS (*Quality of Service*):** Tingkat jaminan pengiriman pesan (QoS 0, QoS 1, QoS 2).
4. **Format JSON:** Memungkinkan penggabungan berbagai parameter telemetri (suhu, kelembapan, status) ke dalam satu paket terstruktur berbasis pasangan *Key-Value*.
5. **Last Will and Testament (LWT):** Fitur keandalan untuk memberi tahu sistem pusat jika sebuah perangkat terputus dari jaringan secara tidak wajar.

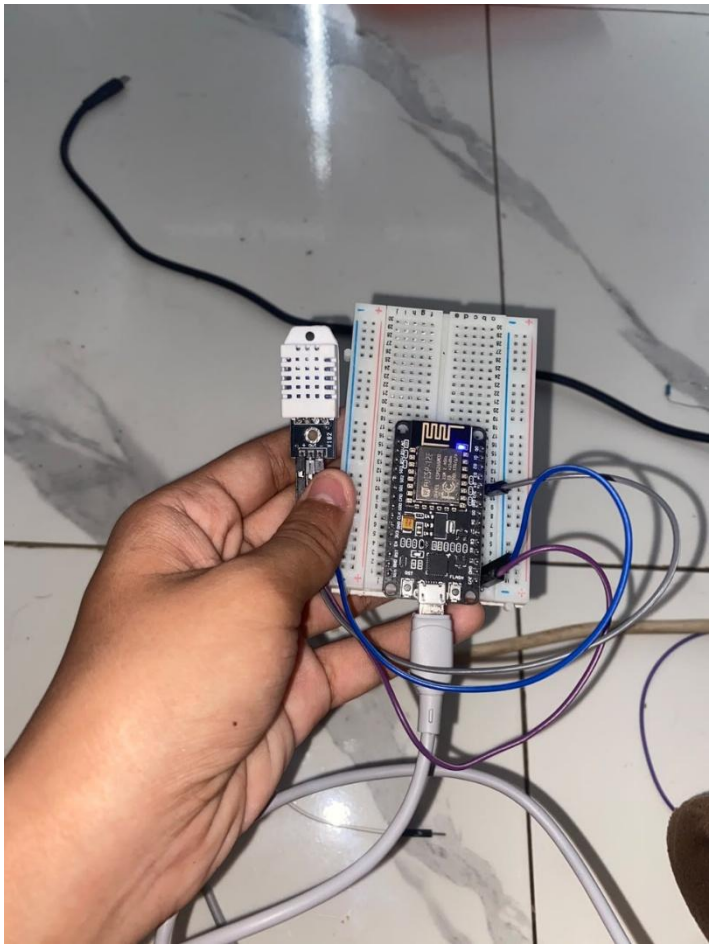
3. Metodologi Percobaan & Kode Program

5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

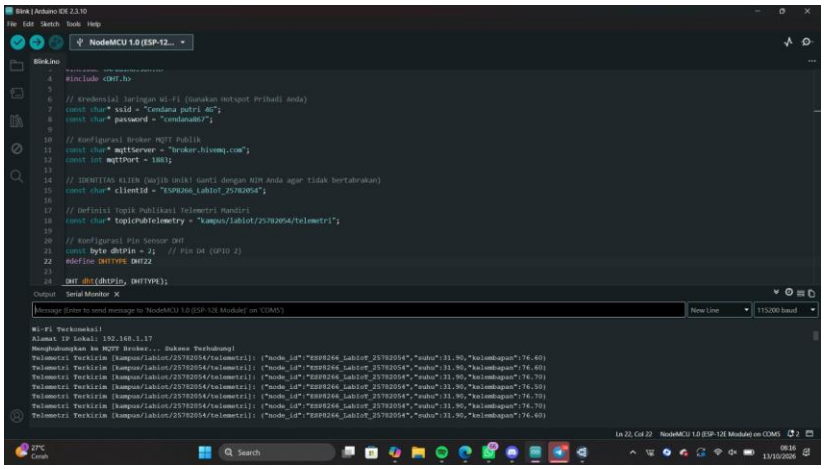
Pada percobaan ini, NodeMCU ESP8266 membaca data dari sensor DHT11 dan mempublikasikannya dalam format JSON setiap 5 detik ke broker publik

(Praktikum 1):

Foto Topologi



ss terminal di prktikum 1



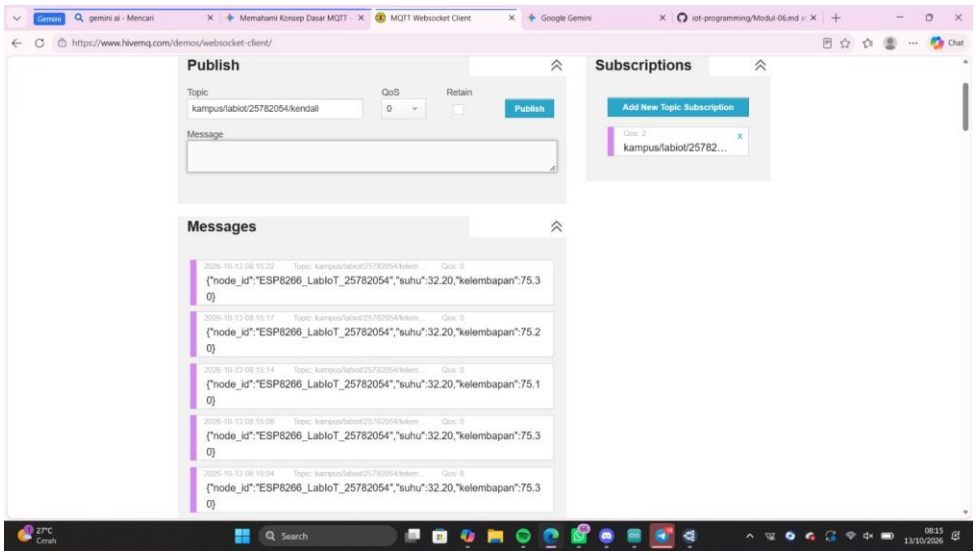
5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (`kampus/labiot/nim_anda/perintah`). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

Langkah Kerja Rangkaian:

1. Pertahankan sensor DHT pada pin D4 dari Praktikum 1.
2. **LED Indikator:** Hubungkan kaki Anoda (kaki panjang) LED ke pin **D1 (GPIO 5)**, dan hubungkan kaki Katoda (kaki pendek) ke Resistor 220 Ω yang menuju ke **GND**.

ss mqttt praktikum 1



Penjelasan Topologi:

1. **Node Publisher & Subscriber (NodeMCU ESP8266):** Membaca data fisik dari sensor DHT11 di pin D4 dan mengirimkan (*Publish*) telemetri berformat JSON ke topik `kampus/labiot/ghalif/telemetri` setiap 4 detik. NodeMCU juga secara

asinkron mendaftarkan diri (*Subscribe*) ke topik `kampus/labiot/ghalif/kendali` untuk mendengarkan perintah kontrol LED.

2. **MQTT Broker (`broker.hivemq.com`):** Berfungsi sebagai pusat distribusi dan perutean pesan antar simpul secara *real-time*.
3. **Diagnostic Tester (MQTT Explorer):** Bertindak sebagai dashboard operator untuk memantau grafik telemetri, menerima sinyal *alert* darurat jika suhu $> 31.0^\\circ\text{C}$, serta mengirimkan perintah manual *override* LED.

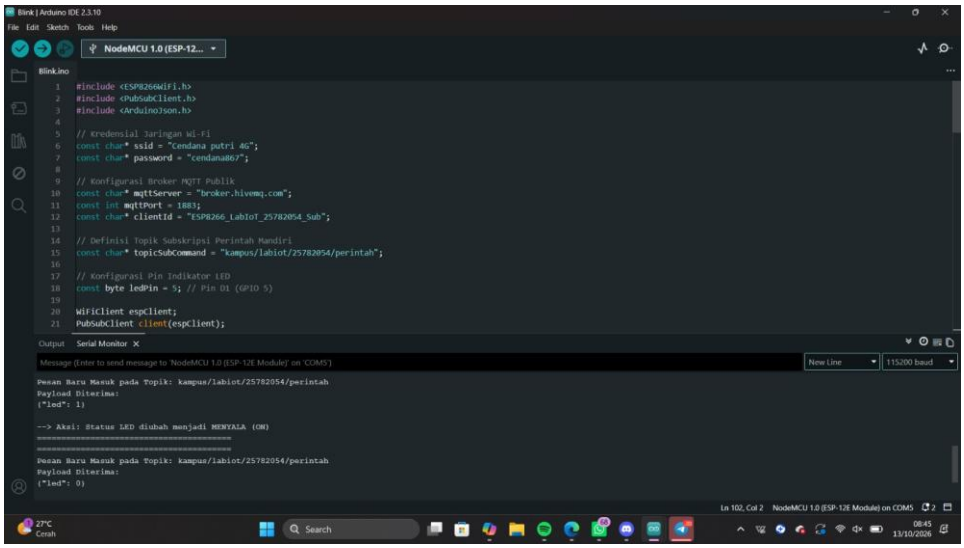
5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (`kampus/labiot/nim_anda/perintah`). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

Langkah Kerja Rangkaian:

1. Pertahankan sensor DHT pada pin D4 dari Praktikum 1.
2. **LED Indikator:** Hubungkan kaki Anoda (kaki panjang) LED ke pin **D1 (GPIO 5)**, dan hubungkan kaki Katoda (kaki pendek) ke Resistor 220 Ω yang menuju ke **GND**.

Output pada Serial Monitor



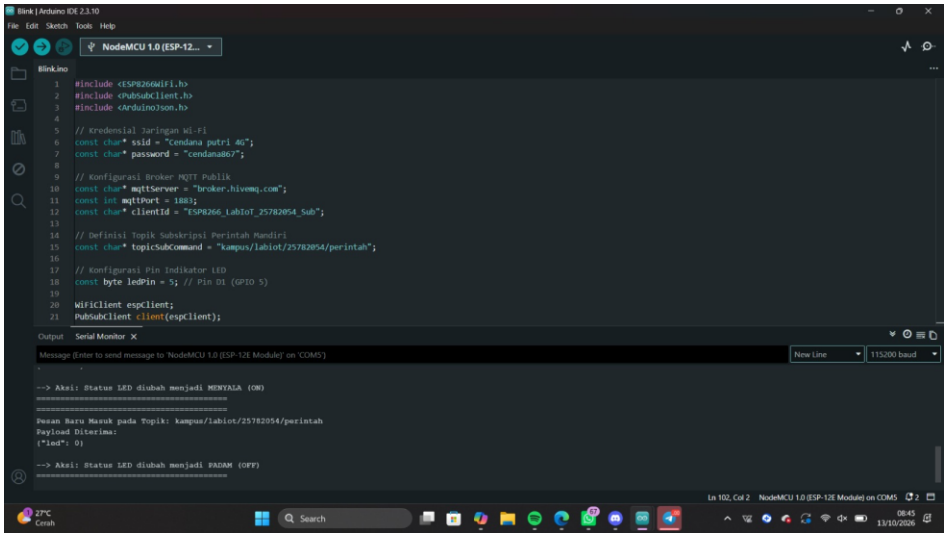
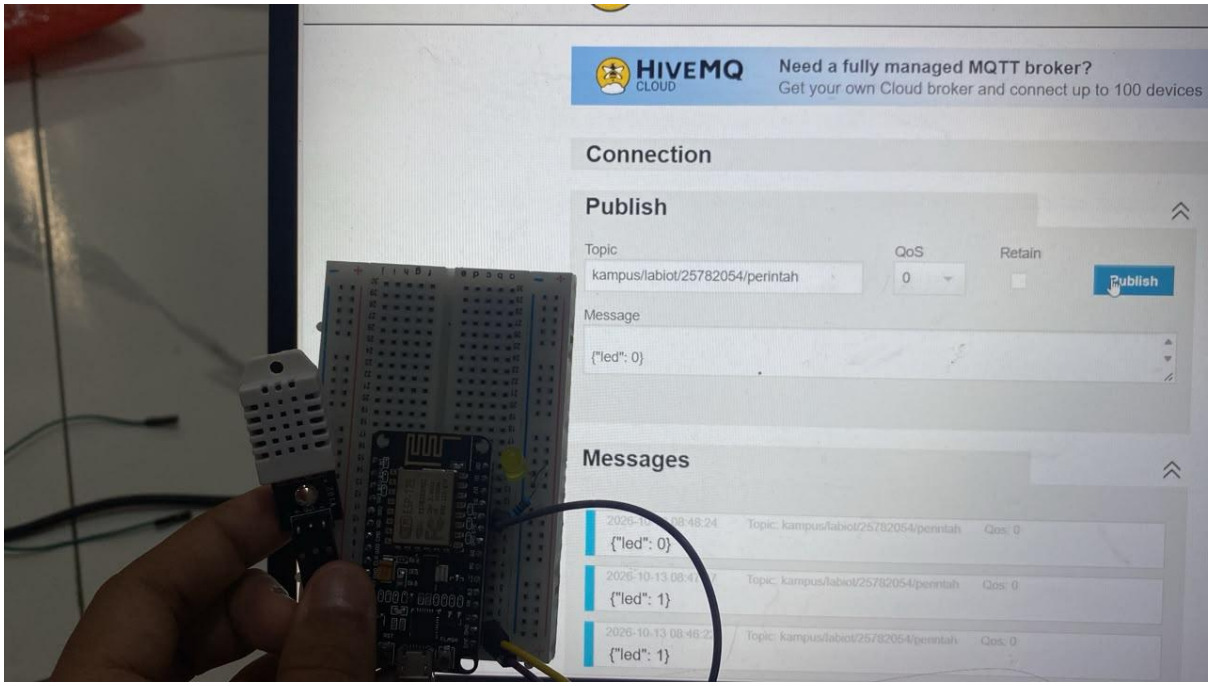




Foto topologinya



- **Fungsi Utama:** Memprogram NodeMCU ESP8266 sebagai ***Subscriber*** yang mendengarkan perintah dari luar. Ketika ada pesan masuk pada topik perintah, mikrokontroler akan memproses data tersebut untuk menyalakan atau mematikan lampu LED secara seketika (asinkron).

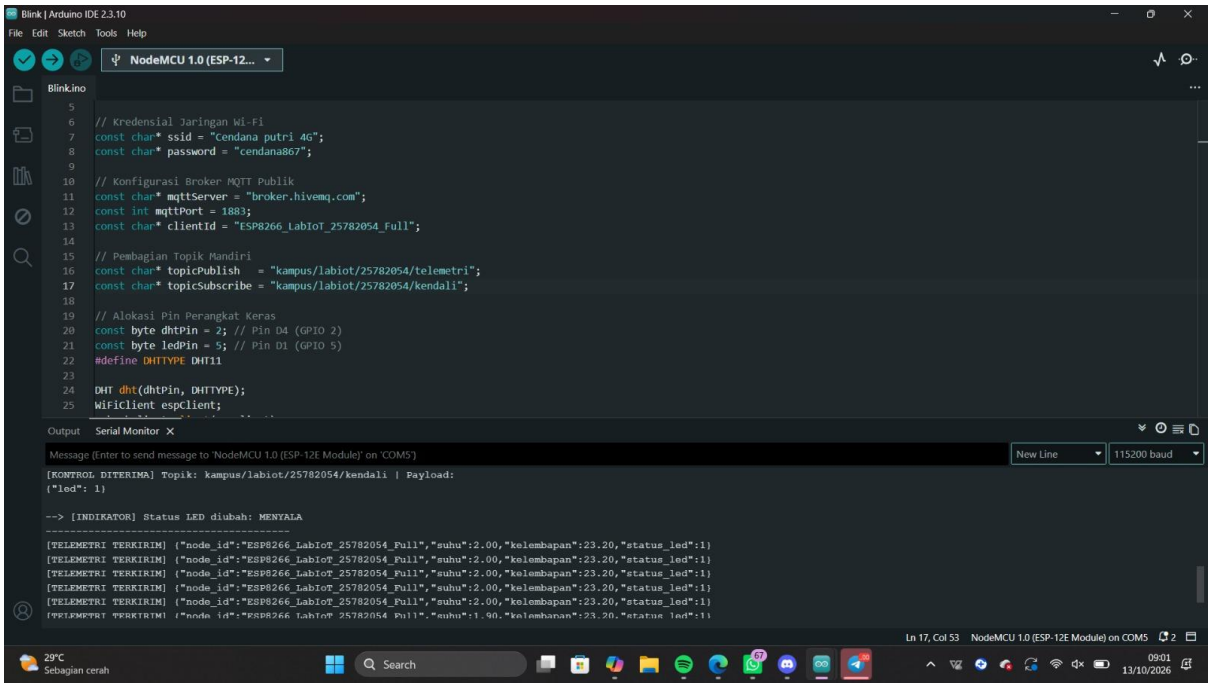
- **Alur Kerja Teknis:**

1. Setelah berhasil terhubung ke broker, klien langsung mendaftarkan diri pada topik perintah melalui fungsi `client.subscribe(topicSubCommand)` di dalam fungsi pemulihan koneksi `reconnectMQTT()`.
2. Ketika broker mengirimkan pesan ke topik tersebut, fungsi khusus bernama ***Callback*** (`onMessageReceived`) akan terpicu secara otomatis tanpa harus dipanggil manual di dalam `loop()`.
3. Di dalam fungsi *callback*, deretan *byte* payload yang masuk diubah menjadi format teks *string* yang aman, lalu diurai (*deserialization*) menggunakan `deserializeJson()`.
4. Program memeriksa kunci `"led"` pada payload JSON (`{"led": 1}` atau `{"led": 0}`). Jika bernilai `1`, pin LED diatur menjadi `HIGH` (menyala); jika `0`, diatur menjadi `LOW` (padam).

5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Pada praktikum ini, kita menyatukan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam sebuah sistem terpadu (*Full-Duplex Telemetry & Control*). NodeMCU secara kontinu memublikasikan data telemetri sensor lingkungan (Suhu, Kelembapan, dan Status LED fisik) ke Broker setiap 4 detik, sembari tetap responsif mendengarkan dan mengeksekusi instruksi kontrol LED dari topik perintah kapan saja.

Output pada Serial Monitornya



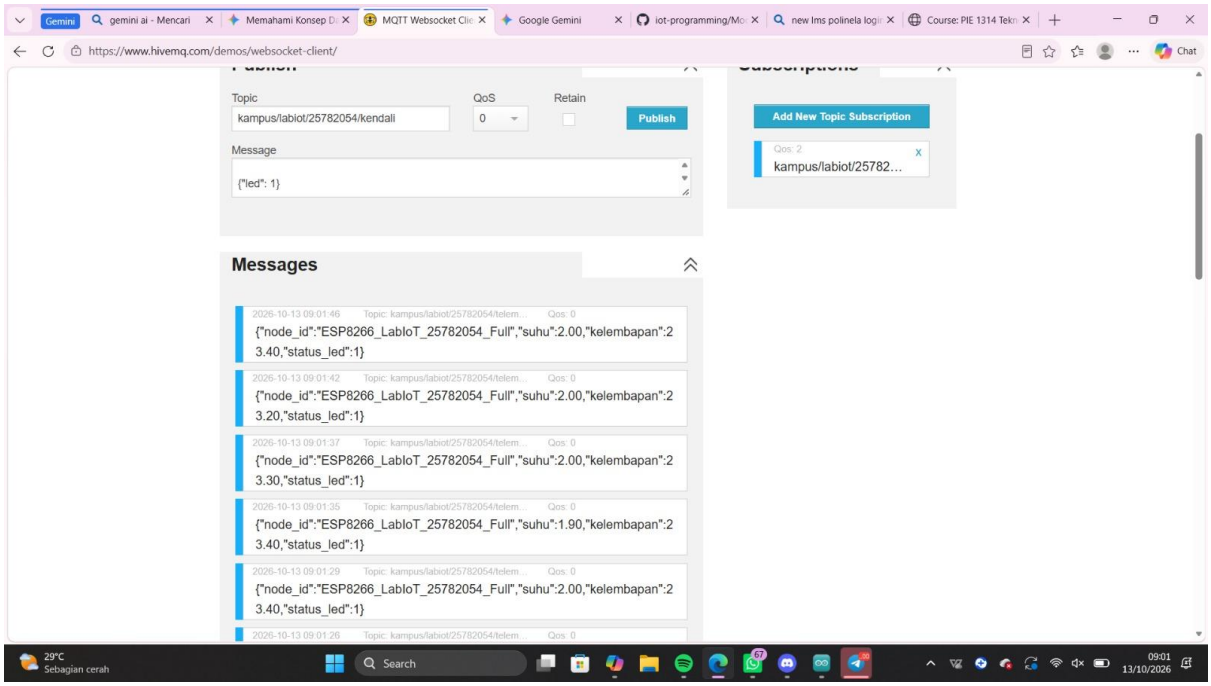
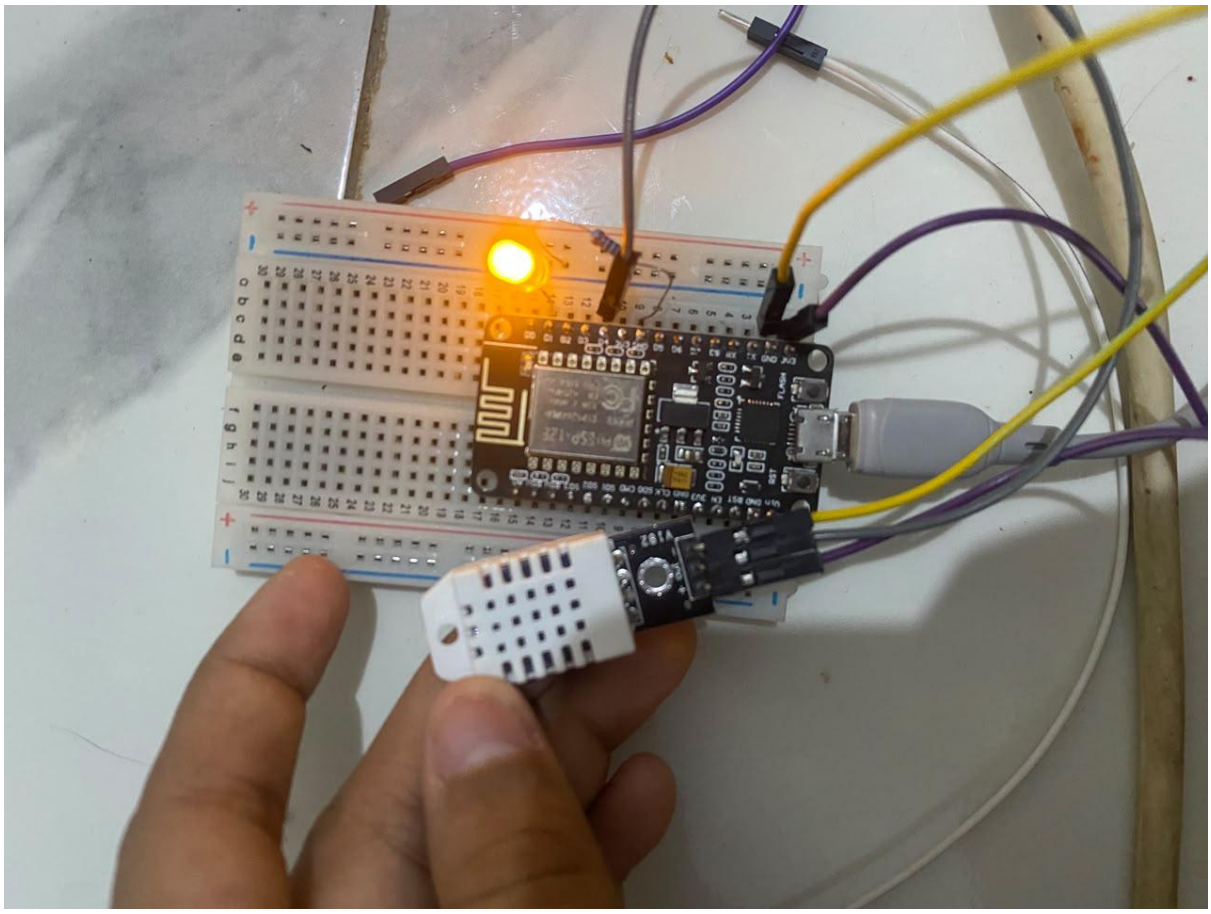


Foto topologinya



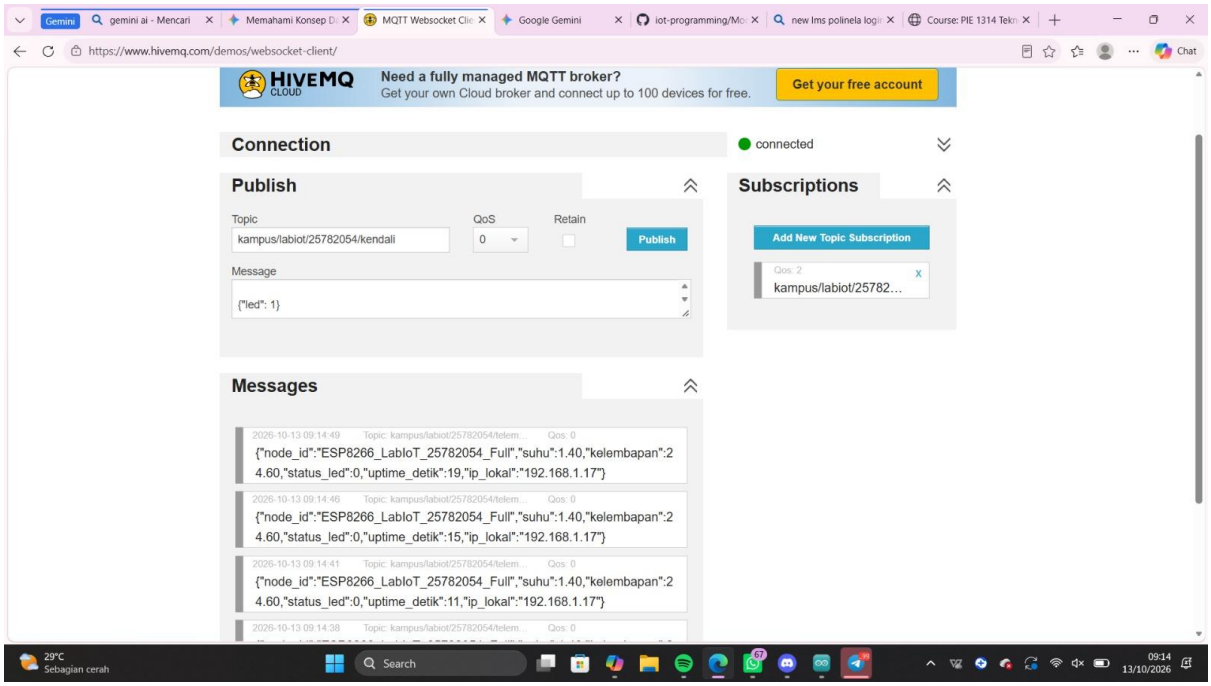
- **Fungsi Utama:** Menggabungkan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam satu sistem terpadu ***Full-Duplex***. NodeMCU bertindak secara simultan sebagai pengirim data telemetri sekaligus penerima perintah kontrol.
- **Alur Kerja Teknis:**
    1. Sistem melakukan inisialisasi sensor DHT dan pin LED pada fungsi `setup()`, serta menghubungkan mikrokontroler ke jaringan Wi-Fi dan broker MQTT.
    2. Di dalam fungsi utama `loop()`, mikrokontroler menjalankan dua tugas utama secara bersamaan:
        - **Tugas 1 (Publisher):** Mengirimkan data telemetri (suhu, kelembapan, dan status LED fisik saat itu) berformat JSON ke topik telemetri setiap 4 detik sekali menggunakan *timer non-blocking*.
        - **Tugas 2 (Subscriber & Listener):** Terus memanggil `client.loop()` untuk memelihara koneksi jaringan sekaligus mendengarkan instruksi kontrol masuk dari topik kendali secara *real-time*.
    3. Jika operator mengirimkan perintah kendali melalui aplikasi eksternal (seperti MQTT Explorer), fungsi *callback* akan langsung mengeksekusi perubahan status LED tanpa mengganggu atau menunda siklus pengiriman data telemetri sensor.

6.6 Bagian Latihan

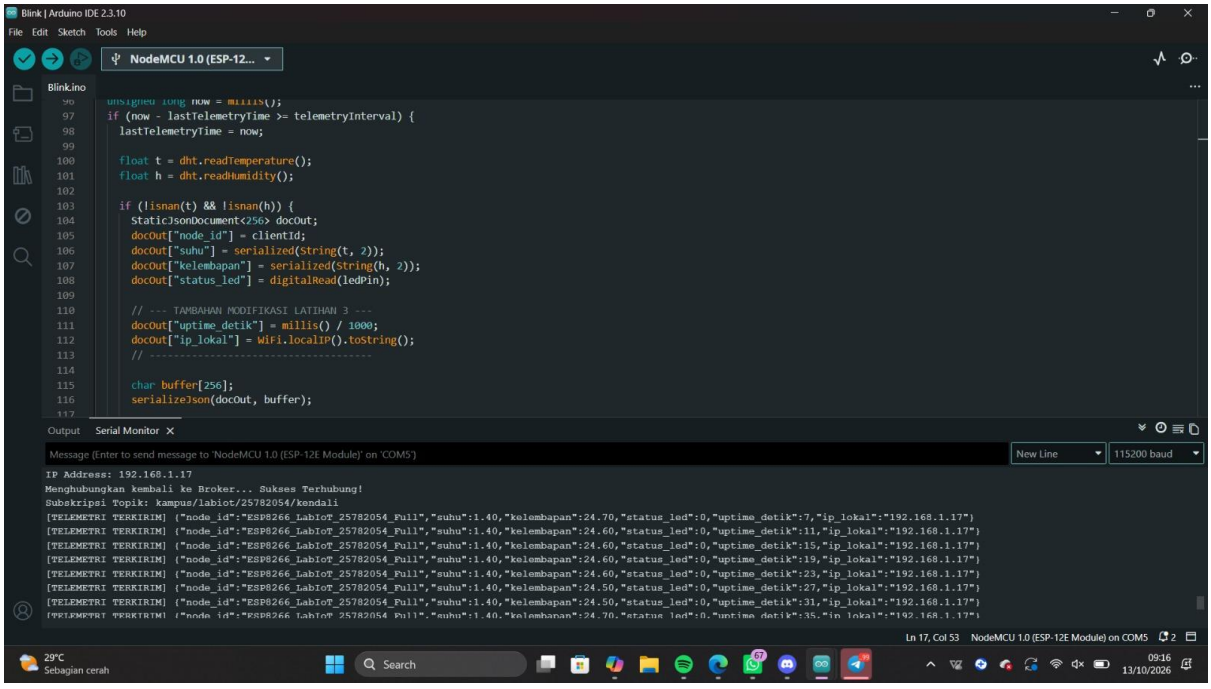
Bagian latihan dirancang untuk menguji pemahaman konseptual dan analitis mahasiswa terhadap materi protokol MQTT:

1. **Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):**
    - **Tujuan:** Meminta mahasiswa membandingkan secara mendalam antara protokol HTTP (yang dipelajari di Modul 3) dengan MQTT.
    - **Poin Pembahasan:**
        - *Bandwidth:* HTTP memiliki *header overhead* yang besar ($\approx 200–800$ byte), sedangkan MQTT sangat ringan (minimal 2 byte), sehingga jauh lebih hemat untuk paket data seluler.
        - *CPU Overhead:* HTTP berbasis *Request-Response* yang boros sumber daya, sedangkan MQTT menggunakan model *Publish-Subscribe* terpusat yang efisien bagi mikrokontroler.
        - *Skalabilitas (1-to-N):* MQTT lebih unggul dalam mendistribusikan data ke banyak perangkat penerima sekaligus melalui perantara broker.
2. **Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):**
    - **Tujuan:** Melatih kemampuan mahasiswa dalam merancang dan memfilter topik MQTT menggunakan karakter pengganti (*wildcard*) `+` (*single-level*) dan `#` (*multi-level*).
    - **Contoh Penyelesaian Konseptual:**
        - Memantau suhu dari seluruh mesin di semua lini (`pabrik/+/+/suhu`).
        - Memantau seluruh parameter sensor khusus mesin `m1` di `line2` (`pabrik/line2/m1/#`).
        - Memantau seluruh lalu lintas data pabrik (`pabrik/#`).
3. **Latihan 3 (Implementasi Serialisasi Data Dinamis):**
    - **Tujuan:** Mempraktikkan penambahan atribut data dinamis ke dalam payload JSON menggunakan pustaka `ArduinoJson`.
    - **Poin Pembahasan:** Mahasiswa diminta menyisipkan parameter tambahan seperti `"uptime_detik"` (diambil dari `millis() / 1000`) dan `"ip_lokal"`

(diambil dari `WiFi.localIP().toString()`) ke dalam struktur dokumen JSON sebelum diserialisasi.



serial monitor



4. **Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):**
    - **Tujuan:** Memahami pentingnya fungsi `client.loop()` dan dampak negatif dari penggunaan fungsi penunda waktu yang memblokir proses (*blocking delay*).
    - **Poin Pembahasan:** Jika pengembang memasukkan `delay(6000)` di dalam `loop()`, mikrokontroler akan berhenti mengeksekusi `client.loop()`.

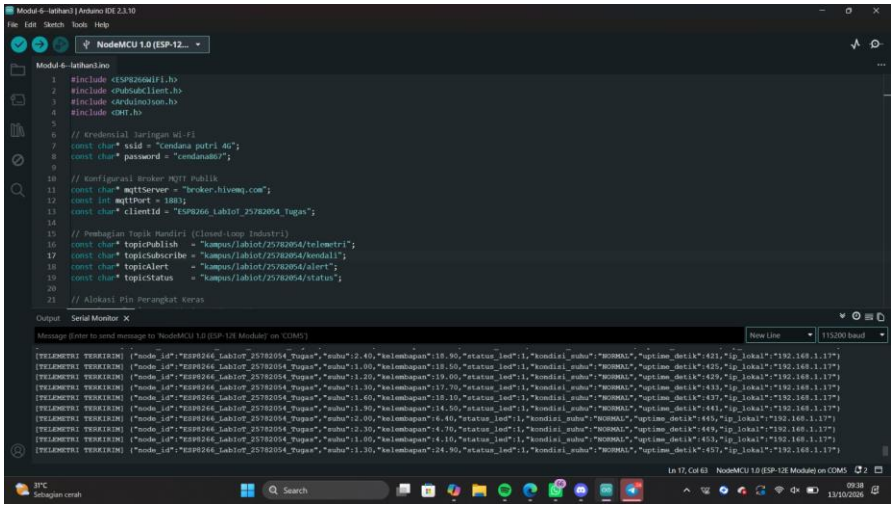
Akibatnya, hubungan *keep-alive ping* dengan broker terputus, pesan *callback* penanganan kendali LED tidak akan direspons (*lag*), dan sistem menjadi tidak stabil.

6.7 Tugas (Sistem Telemetri & Kendali Closed-Loop Industri)

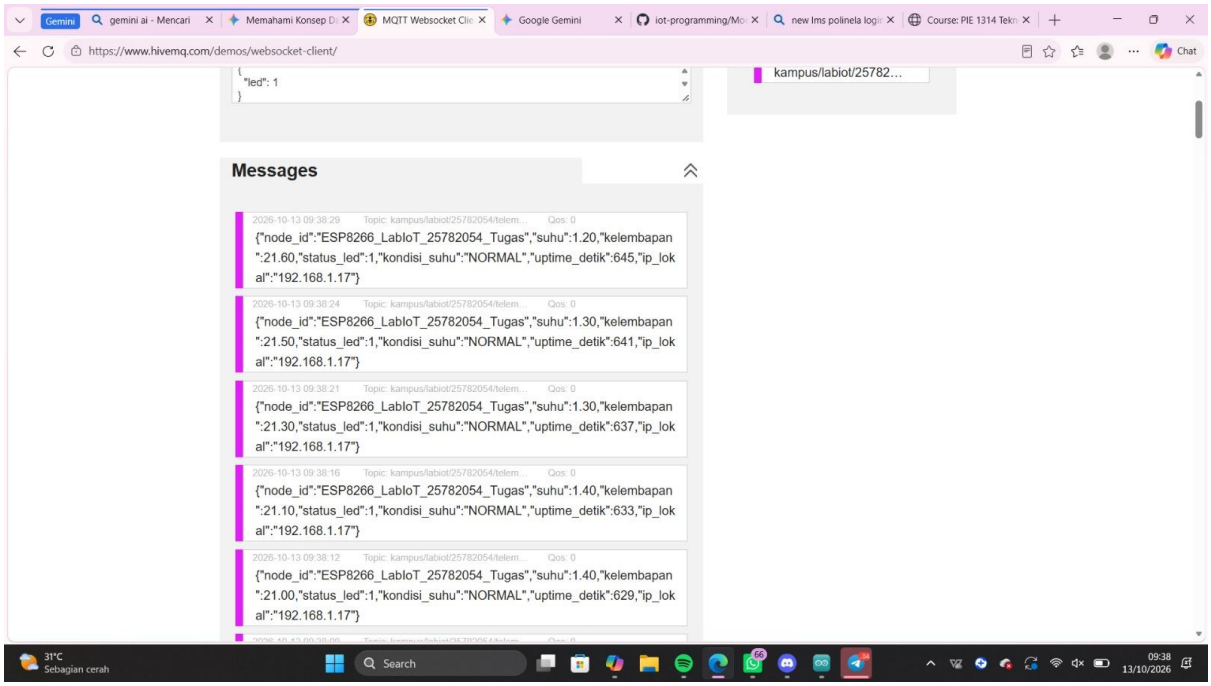
Bagian tugas merupakan proyek komprehensif tingkat lanjut yang menggabungkan seluruh materi modul menjadi sebuah sistem otomatisasi industri tertutup (*Closed-Loop Industrial Safety*):

- **Arsitektur & Alur Sistem:**
    1. **Rule Engine Lokal (Edge Intelligence):** NodeMCU membaca sensor suhu secara kontinu. Jika suhu melebihi ambang batas aman ($> 31.0^\circ\text{C}$), mikrokontroler tidak hanya mengirim data telemetri biasa, tetapi juga langsung memicu status `"PANAS"` dan mengirimkan paket peringatan darurat mandiri (*dedicated alert*) ke topik `.../alert`.
    2. **Otomasi Aktuator Darurat:** Ketika kondisi *overheat* terdeteksi, sistem secara otomatis menyalakan LED indikator bahaya tanpa harus menunggu perintah dari operator luar.
    3. **Last Will and Testament (LWT):** Mendaftarkan pesan wasiat ke broker saat awal koneksi. Jika mikrokontroler terputus secara tidak wajar (misalnya kabel daya dicabut), broker secara otomatis menyiarkan status `OFFLINE` ke sistem pemantau pusat.
    4. **Manual Override:** Selama kondisi suhu normal, operator tetap dapat mengontrol LED secara manual jarak jauh melalui topik kendali (`.../kendali`) menggunakan MQTT Explorer.
- **Luaran yang Diharapkan:** Kode program `.ino` yang terstruktur, dokumentasi pengujian fisik, tangkapan layar MQTT Explorer yang menampilkan alur data telemetri, respons *alert*, serta uji coba pencabutan daya LWT yang diunggah ke repositori GitHub pribadi.

serial monitor tugas



MQTT



Analisis Kriteria Keberhasilan Tugas Praktikum

Pada bagian akhir modul, sistem Anda harus memenuhi parameter-parameter pengujian berikut agar dinyatakan berhasil:

- **Stabilitas Telemetri & Klasifikasi Kondisi:** Data sensor terkirim secara periodik dan stabil, lengkap dengan atribut string tambahan berupa `"kondisi_suhu"` yang bernilai `"NORMAL"` atau `"PANAS"` berdasarkan ambang batas yang ditentukan (31.0°C).
- **Respon Peringatan Dini (*Alert Topic*):** Ketika sensor DHT mendeteksi suhu di atas 31.0°C, mikrokontroler secara otomatis memublikasikan paket peringatan (*overheat*) ke topik `alert` sekaligus menyalakan LED indikator bahaya secara mandiri tanpa menunggu intervensi operator.
- **Kendali Manual (*Manual Override*):** Operator dari luar tetap dapat mengambil alih atau mengubah status LED secara manual melalui topik `kendali` saat kondisi suhu dalam keadaan aman.
- **Keandalan Koneksi (LWT):** Fitur *Last Will and Testament* (LWT) terbukti berfungsi ketika kabel daya atau koneksi mikrokontroler terputus secara mendadak, di mana *broker* akan otomatis menyiarkan pesan status `OFFLINE`.
- **Kualitas Kode Program:** Kode ditulis bersih tanpa menggunakan fungsi penunda waktu yang memblokir proses sistem (`delay()`), serta dilengkapi dengan komentar dokumentasi teknis yang jelas.

Rangkuman Modul 6

Rangkuman ini menyimpulkan esensi dari keseluruhan modul:

- **Adopsi Standar Industri:** Modul ini memperkenalkan standar komunikasi industri modern menggunakan **MQTT (*Message Queuing Telemetry Transport*)**.
- **Arsitektur Efisien:** Berkat arsitektur terpusat *Publish-Subscribe* dan bantuan *MQTT Broker*, NodeMCU ESP8266 terbebas dari kerumitan manajemen sesi klien individual, sehingga pengunaan *bandwidth* dan daya menjadi sangat efisien.
- **Format Data & Keandalan:** Penggunaan pustaka `ArduinoJson` memungkinkan pengemasan data sensor menjadi terstruktur. Dipadukan dengan pengiriman *non-blocking*, *callback* asinkron, *Rule Engine* lokal, serta proteksi darurat LWT, Anda telah berhasil membangun simpul IoT kelas industri yang tangguh dan siap terhubung ke sistem *cloud*.